CSE 445 — Computer Vision

Assignment: Semantic Segmentation Model Benchmarking

Part 1 of 2 — Dataset Analysis, Training & Evaluation

Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

Dr. Md Rifat Ahmmad Rashid
Associate Professor, Dept. of CSE
rifat.rashid@ewubd.edu

Summer 2026

Mode: Group (dataset pre-assigned) **Deadline:** **14.07.2026, 11:59 PM**

Evaluation: Kaggle Notebooks (public links) submitted via Google Classroom, followed by Viva Voce

Models:
- DeepLabV3 (ResNet backbone)
- SegFormer-B0 (MiT-B0 encoder)
- YOLOv26 Semantic (`-sem` checkpoints)

---

1. Overview

Note — This is a Two-Part Assignment

This document specifies **Part 1** only: each group benchmarks **three** semantic segmentation approaches on their assigned dataset, covering the full pipeline from EDA through final model comparison. **Part 2 will be announced separately** on Google Classroom after the Part 1 deadline.

1.1. Learning Objectives

By the end of Part 1, each student should be able to:

- Perform structured Exploratory Data Analysis (EDA) on a segmentation dataset (image + mask pairs).
- Design a train/validation/test split that is provably free of data leakage.
- Apply segmentation-appropriate data augmentation correctly (synchronized image–mask transforms, train-only).
- Train and fine-tune three architecturally distinct segmentation models: a CNN/ASPP-based segmenter, a hierarchical vision-transformer segmenter, and a real-time YOLO-based semantic segmenter.
- Evaluate models using standard segmentation metrics and perform structured error analysis.
- Defend every coding and hyperparameter choice in a live viva examination.

1.2. Dataset

Each group has been pre-assigned a unique dataset. Before writing any model code, confirm and document:

- Number of images, number of classes, class names / IDs.
- Mask format (single-channel class-index PNG, RGB-coded mask, COCO-style polygon, etc.) and how pixel values map to class IDs.
- Image resolution(s) and whether they are uniform across the dataset.
- Whether an official train/test split is provided, or whether you must create your own.

2. Required Tasks

2.1. Task A — Exploratory Data Analysis

- Compute class distribution at the **pixel level** (pixel-count per class, not just image-count) — pixel-level imbalance is what drives loss-function and class-weighting decisions.
- Plot the image size and aspect-ratio distribution.
- Display representative image–mask pairs for every class.
- Check for near-duplicate images, corrupted files, and mismatched image–mask pairs.
- Summarize the degree of class imbalance and state how it will influence your loss-function choice in Task E.

2.2. Task B — Leakage-Safe Data Splitting

Split into **Train / Validation / Test** (70/15/15 is a reasonable default; justify your own ratio if you deviate).

Critical — Preventing Data Leakage

- If the dataset contains grouped images (same source subject, scene, video, or multiple crops of a single original), split **by group**, not by individual image.
- Augmented images must stay in the **same split** as their source; augmentation is always generated *after* splitting, never before.
- Fix a `random_state` / seed and report it so the split is fully reproducible.
- Save the split (file lists or indices) once and **reuse the identical split across all three models** — this is mandatory for a fair benchmark.

2.3. Task C — Data Augmentation

- Use a library that supports synchronized image+mask transforms (e.g., Albumentations).
- Apply augmentation to the **training split only** — never to validation/test.
- Select augmentations appropriate to your dataset domain (e.g., avoid vertical flips for orientation-sensitive scenes such as street views; they may be fine for top-down or medical imagery). **Justify each transform and its probability.**
- Report the complete augmentation pipeline in a clearly labelled notebook cell.

2.4. Task D — Sanity-Check Visualization

Before writing any training code, produce a grid of **post-augmentation training samples with mask overlays** to confirm:

- Image and mask are correctly aligned (no shift or flip mismatch).
- The class-colour mapping is correct and consistent.
- Augmentations look visually reasonable (no destroyed mask regions).

This visualization must appear in the notebook **before** the first training cell. It is a required quality gate, not optional decoration.

2.5. Task E — Model Training (minimum 50 epochs each)

Approach	Library / Reference	Backbone / Checkpoint	Epochs
DeepLabV3	`torchvision.models.segmentation`	ResNet-50 or ResNet-101 (ASPP head)	50
SegFormer-B0	HuggingFace `transformers`, `SegformerForSemanticSegmentation`	MiT-B0 encoder	50
YOLOv26 (Semantic)	Ultralytics, `-sem` checkpoints	`yolo26n-sem.pt` or larger	50

General requirements for every model:

- Pretrained backbones (ImageNet / COCO / Cityscapes) are allowed and encouraged for transfer learning; fine-tune on your assigned dataset.
- 50 epochs is a **minimum floor**. If a model is still improving on validation mIoU at epoch 50, continue training and report the actual final epoch count.
- In each model notebook, collect all hyperparameters into a single clearly visible config cell and report: optimizer, learning rate (and schedule if used), batch size, input image size, loss function, number of classes, class weights (if applied), and total training time.

2.6. Task F — Testing & Quantitative Evaluation

Evaluate every model on the held-out **test split** (never seen during training or validation) and report:

- Mean IoU (mIoU) and per-class IoU
- Overall Pixel Accuracy and Mean Pixel Accuracy
- Dice coefficient (F1 score at pixel level)
- Pixel-level confusion matrix across all classes

2.7. Task G — Error Analysis

- Find the worst-performing test images for each model (lowest per-image IoU) and visualize prediction vs. ground truth side-by-side.
- Identify which class pairs are most confused and hypothesize why (visual similarity, severe class imbalance, small object size, boundary ambiguity, etc.).
- Discuss whether the three models fail on the **same** images and classes or on **different** ones — this reveals each model's distinct failure modes.

2.8. Task H — Final Comparison

- A summary table comparing all three models on every metric from Task F.
- A bar or radar chart comparing mIoU and at least one other metric across the three models.
- Side-by-side qualitative grids (same test images, all three models + ground truth) for at least 5 representative samples.
- A written verdict: which model performed best, which was most efficient, and which you would deploy in practice — and why.

3. Approved Model Catalogue

How to Choose a Variant

For **DeepLabV3** and **YOLOv26**, your group may select any checkpoint size from the catalogue below — choose based on your dataset size and Kaggle GPU memory (T4/P100, ∼16 GB). For **SegFormer**, the encoder size is fixed at B0 by this assignment; your only choice is which pretrained head to start from. Whatever you choose, report it explicitly in your training configuration cell — size and checkpoint selection are among the first things you will be asked to justify at viva.

3.1. DeepLabV3 — `torchvision.models.segmentation` (any backbone)

Variant	Constructor	Params (M)	Recommended For
DeepLabV3-MobileNetV3	`deeplabv3_mobilenet_v3_large`	∼11	Small datasets, limited compute
DeepLabV3-ResNet50	`deeplabv3_resnet50`	∼42	Default balanced choice
DeepLabV3-ResNet101	`deeplabv3_resnet101`	∼61	Larger datasets, more compute

All three load COCO-pretrained weights via `weights=DeepLabV3_..._Weights.DEFAULT`. Replace the classifier head to match your class count and fine-tune end-to-end (or backbone-frozen for the first few epochs if GPU memory is tight).

3.2. SegFormer-B0 — HuggingFace `transformers` (encoder fixed at B0)

Pretrained Checkpoint (HuggingFace Hub)	Pretrained Domain	Notes
`nvidia/segformer-b0-finetuned-ade-512-512`	ADE20K (150 cls, mixed scenes)	General-purpose default
`nvidia/segformer-b0-finetuned-cityscapes-1024-1024`	Cityscapes (19 cls, urban driving)	Best for street/road data
`nvidia/mit-b0`	ImageNet-1k (encoder only)	Use if no head transfers

Load with: `SegformerForSemanticSegmentation.from_pretrained(checkpoint, num_labels=<N>, ignore_mismatched_sizes=True)`

3.3. YOLOv26 Semantic Segmentation — Ultralytics (any size)

Checkpoint files use the dedicated `-sem` suffix (**not** `-seg`, which is the instance-segmentation task). Task is inferred automatically from the checkpoint name.

Checkpoint	Params (M)	Cityscapes mIoU	Recommended For
`yolo26n-sem.pt`	1.6	78.3	Fastest; small datasets, limited GPU
`yolo26s-sem.pt`	6.5	80.8	Balanced default
`yolo26m-sem.pt`	14.3	82.0	Larger datasets
`yolo26l-sem.pt`	17.9	82.9	More compute available
`yolo26x-sem.pt`	40.2	83.6	Maximum accuracy, heaviest

`Usage: model = YOLO("yolo26n-sem.pt") → model.train(data="data.yaml", epochs=50, imgsz=...)`

4. Required Notebook Structure

Each group submits **4 Kaggle notebooks**, all sharing the **identical split** produced in Notebook 0.

NB	Title	Contents
0	`eda-and-data-prep`	Task A (EDA), Task B (leakage-safe split — save artifact), Task C (augmentation pipeline), Task D (sanity visualization)
1	`seg-deeplabv3`	Load split from NB0 → Task E (DeepLabV3, ≥50 epochs) → Task F → Task G
2	`seg-segformer-b0`	Load split from NB0 → Task E (SegFormer-B0, ≥50 epochs) → Task F → Task G
3	`seg-yolov26-semantic`	Load split from NB0 → Task E (YOLOv26-Sem, ≥50 epochs) → Task F → Task G → **Task H** (final 3-model comparison, pulling saved metrics from NB1–NB2)

Each model notebook (NB1–NB3) must follow this internal order:

1. Setup & imports (with version pins)
2. Load the shared data split from Notebook 0
3. Model definition / pretrained-weight loading
4. Training configuration — all hyperparameters in one clearly visible dict/cell
5. Training loop with per-epoch loss and mIoU curves plotted
6. Test-set evaluation (all Task F metrics)
7. Error-analysis visualizations (Task G)
8. Markdown summary cell (also appends results to the shared `results.json` used by NB3 for Task H)

5. Submission Instructions

1. Run each notebook **end-to-end with all cell outputs visible** before submission.
2. Set each Kaggle notebook to **Public**.
3. Submit all 4 links in a single post to **Google Classroom** in this format:

```
Group:  <group number/name>
Dataset:  <dataset name>
NB0 -- EDA & Data Prep:  <kaggle link>
NB1 -- DeepLabV3:  <kaggle link>
NB2 -- SegFormer-B0:  <kaggle link>
NB3 -- YOLOv26 (Semantic) + Final Comparison:  <kaggle link>
```

Deadline: 13.07.2026, 11:59 PM. Late submissions are subject to the course late-policy unless otherwise announced.

6. Viva Voce — Format and Expectations

6.1. Expected Coding Questions

Be prepared to walk through and justify, line by line if asked:

6.1.1. Data Pipeline (NB0)

- How did you load and parse the mask format for your specific dataset?
- Walk through your train/val/test split code. How did you verify no image appears in more than one split? What seed did you use and why?
- Show and explain each Albumentations transform: what does it do geometrically or photometrically, why did you include it, and what probability did you set?
- How does your `__getitem__` method ensure the image and mask receive the same random transform at every call?
- What does your sanity-check visualization confirm, and what would a bug look like?

6.1.2. DeepLabV3 (NB1)

- Which torchvision constructor did you use and why? What does changing the backbone (MobileNetV3 vs. ResNet50 vs. ResNet101) trade off?
- How did you adapt the pretrained classifier head for your class count?
- What is the ASPP module and why does it help with multi-scale scene understanding?
- What loss function did you use? If you applied class weights, how did you compute them from the EDA pixel-count histogram?
- What optimizer and learning-rate schedule did you use? What does the schedule do to the LR at each step?
- Walk through your training loop: how do you switch between `model.train()` and `model.eval()`, and why does it matter?

6.1.3. SegFormer-B0 (NB2)

- What pretrained checkpoint did you load and why did you choose that domain?
- Explain `ignore_mismatched_sizes=True`: which layer(s) are re-initialized and why?
- What is the Mix Transformer (MiT-B0) encoder? How does it differ from a standard ViT?
- What is the All-MLP decoder head in SegFormer? How does it fuse features from multiple encoder stages?
- How did you handle the HuggingFace loss output vs. computing your own loss? Which approach did you take and why?
- How did you resize or pad images to the required resolution, and was that done inside the model or in the dataloader?

6.1.4. YOLOv26 Semantic (NB3)

- What is the difference between a `-sem` and a `-seg` checkpoint? Why does using the wrong one change the task entirely?
- What does your dataset YAML file contain? Walk through each field.
- What mask format does YOLOv26-sem expect (pixel value = class ID, ignore value 255)? How did you verify your masks conform to this?
- What does the `imgsz` argument control and how did you pick your value given the dataset resolution?
- How do you read back `result.semantic_mask.data` and convert it to mIoU?

- What optimizer and LR schedule does Ultralytics use by default for semantic training, and did you override any of them?

6.1.5. Error Analysis & Comparison (NB3 Task H)

- Which model achieved the best mIoU on your dataset? Why do you think that model outperformed the others for this specific data?
- Which class was hardest across all three models? What visual or statistical reasons explain this?
- Do the three models fail on the same images or different ones? What does that tell you about architectural differences?
- If you had to improve the worst-performing model, what specific change would you make (architecture, loss, augmentation, learning rate) and why?

6.2. Expected Theory Questions

Be ready to explain the following concepts clearly and concisely:

6.2.1. Semantic Segmentation Fundamentals

- What is semantic segmentation and how does it differ from object detection, instance segmentation, and panoptic segmentation?
- What is the Intersection over Union (IoU) metric? Derive the formula and explain why mIoU is preferred over pixel accuracy for imbalanced datasets.
- What is the Dice coefficient? Show its relationship to the F1 score and IoU.
- Why does a confusion matrix computed at the pixel level differ from one computed at the image level?
- What is the "ignore index" (value 255 in many datasets) and why is it needed during loss computation?

6.2.2. Encoder–Decoder Architectures

- Explain the encoder–decoder structure. What role does the encoder play and what does the decoder recover?
- What is a skip connection (as in U-Net)? How does it help preserve spatial detail lost during downsampling?
- What is atrous (dilated) convolution? Draw a diagram and explain how the dilation rate controls the receptive field without increasing parameters.
- What is the ASPP (Atrous Spatial Pyramid Pooling) module in DeepLabV3? Why use multiple dilation rates in parallel?
- What does "output stride" mean in DeepLabV3 and how does reducing it affect segmentation resolution vs. computation?

6.2.3. Transformer-Based Segmentation (SegFormer)

- What is a Vision Transformer (ViT)? How does it divide an image into patches and process them?
- What is the Mix Transformer (MiT) and how does it differ from a standard ViT (e.g., overlapping patch embedding, hierarchical feature maps, efficient self-attention)?
- Why does SegFormer use a lightweight All-MLP decoder instead of a heavy CNN decoder? What are the advantages?
- Explain multi-head self-attention in the context of image patches. What does the attention map visualize?

- What does "B0" signify in SegFormer-B0 and how would B2 or B5 differ in capacity and computational cost?

6.2.4. YOLO-Based Semantic Segmentation

- YOLO was originally an object detector. Explain how the architecture is adapted for semantic segmentation (per-pixel output rather than bounding boxes).
- What is the C2f (Cross Stage Partial with 2 convolutions) block used in YOLO26? How does it improve gradient flow?
- What is the YOLO neck (PAN / FPN)? Why is multi-scale feature fusion important for segmentation?
- Why does YOLOv26-sem use `imgsz=1024` by default (trained on Cityscapes at 1024×2048)? What trade-off does a smaller `imgsz` introduce at fine-tuning time?
- What does the `-sem` checkpoint output compared with a `-seg` checkpoint? Why are the evaluation metrics different for each task?

6.2.5. Training & Optimization

- What is the cross-entropy loss for segmentation? Write the pixel-wise formula.
- What is Focal Loss? When is it preferred over cross-entropy and why?
- How do you compute class weights from pixel-frequency counts to handle class imbalance? Give the formula.
- Compare SGD with momentum vs. Adam vs. AdamW for fine-tuning a pretrained segmentation model. What does weight decay do?
- What is a learning-rate warmup and cosine annealing schedule? Sketch the LR curve and explain when each phase activates.
- What is transfer learning? Why is it almost always better to start from a pretrained backbone rather than random initialization for segmentation?
- What is overfitting and how do you detect it from training vs. validation loss/mIoU curves? Name three techniques to mitigate it.

6.2.6. Data & Augmentation

- What is data leakage in the context of a segmentation benchmark? Give one concrete example specific to segmentation datasets (e.g., video frames, tiled images).
- Why must image and mask receive the **same** geometric transform but the mask must **not** receive colour-jitter transforms? What goes wrong otherwise?
- What is random horizontal flip, random crop, colour jitter, and Gaussian blur? Which of these are safe to apply to masks and which are not?
- What is Albumentations? How does it guarantee synchronized image–mask transforms?
- What are the three canonical data split roles (train/val/test) and what is each used for? Why must the test set be touched only once?

7. Suggested Marking Scheme

(editable — confirm final weights before distributing to students)

Component	Marks
EDA (Task A)	10
Leakage-safe Split (Task B)	10
Augmentation (Task C)	10
Sanity Visualization (Task D)	5
Training — 3 models (Task E)	30
Testing & Metrics (Task F)	15
Error Analysis (Task G)	10
Final Comparison (Task H)	10
Notebook Submission Subtotal	100

Viva Voce: weighted separately per course policy. Individual marks may differ within a group based on each member's ability to explain their own code and parameter choices.

8. Academic Integrity

- All code must be your group's own work. You may reference official documentation (Ultralytics, HuggingFace, PyTorch/torchvision) and must cite it in markdown cells.
- Identical notebooks or near-identical results across different groups will be investigated as an academic integrity concern.
- Inability to explain your own code at viva will reduce your individual viva mark regardless of notebook output quality.

Part 1 — Final Submission Checklist

1. **NB0**: EDA complete → leakage-safe split saved → augmentation pipeline defined → sanity visualization shown
2. **NB1**: DeepLabV3, ≥50 epochs, test metrics + error analysis
3. **NB2**: SegFormer-B0, ≥50 epochs, test metrics + error analysis
4. **NB3**: YOLOv26-Sem, ≥50 epochs, test metrics + error analysis + **final 3-model comparison** (Task H)
5. All 4 notebooks set to **Public** on Kaggle
6. All 4 links submitted together in **Google Classroom**
7. **Deadline: 13.07.2026, 11:59 PM** — Viva to follow

References:

[1] Ultralytics, *YOLO26 Semantic Segmentation.* `https://docs.ultralytics.com/tasks/semantic`

[2] HuggingFace, *SegFormer Model Documentation.* `https://huggingface.co/docs/transformers/model_doc/segformer`

[3] PyTorch, *torchvision DeepLabV3 Documentation.* `https://pytorch.org/vision/stable/models/deeplabv3.html`

[4] Xie et al., *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers.* NeurIPS 2021.

[5] Chen et al., *Rethinking Atrous Convolution for Semantic Image Segmentation (DeepLabV3).* arXiv:1706.05587, 2017.

Next: Part 2 will be announced on Google Classroom after the Part 1 deadline.